CSC 4553

Fall 2026 - Day 17

Admin

- Lab due Friday, 21:00
- Quiz due Monday, 21:00
- Mid-semester survey on Canvas

Memory Optimizations

Questions to consider

- How can we reduce memory footprint without limiting functionality?
- What are the benefits and challenges of sharing code across processes?
- What are the trade-offs in swapping processes to disk?

Dynamic loading

- Load parts of a process only when needed
  - → Why?
    - Otherwise processes must fit entirely in physical memory
- Granularity: per function or per module (shared library)
- Smaller footprint, faster startup, more complexity
- Where would this help most?
  - → Rarely used code (error handlers), plugins

Dynamic linking

- Link libraries at runtime instead of compile time
- Processes share one copy of library code in memory
- Libraries can be updated without recompiling
- Needs OS + dynamic linker/loader support
- What could go wrong?
  - → Missing or incompatible library: fails to start or crashes
  - → Library updated while in use

Swapping

- Move an entire process from memory to a backing store (disk)
- Q: Why swap?
  - → Run more processes than RAM can hold
- Q: When do we swap?
  - → When processes don’t all fit in physical memory
- Q: When swapping back in, where does the process go?
  - → Depends on memory binding
  - → Runtime binding: anywhere

Pick the victim

- Memory is full and P2 needs to run
- You must swap a process out
- What would you want to know about each process before choosing?

Swapping caveats

- Disk is slow: be careful who, how, and when we swap
  - → Swap time is proportional to memory swapped
  - → Transfer time dominates, not context switching
- Process state matters
  - → E.g., processes blocked on I/O
- In reality: we swap **pages**, not whole processes (more later)

What isn’t clear?

Comments? Thoughts?

Contiguous allocation

Questions to consider

- How do we keep track of free memory and decide where to place processes?
- What problems arise from fragmentation, and how can we address them?
- What are the trade-offs between different allocation strategies?

Memory allocation strategies

- How do we find free memory for processes?
- How do we keep track of free memory?
- Which free block do we give to a process?

Contiguous allocation

- Memory is allocated in contiguous blocks
- *Fixed-sized partitions*: one process each
  - → Simple but inflexible
- *Variable-sized partitions*: allocate exactly what's needed

Variable partitioning

- Any obvious problems?
  - → External fragmentation: small free blocks that are too small to use
- Allocate processes until nothing fits in any “hole”
- Then what?
  - → Rearrange memory: compaction
    - This is **expensive**

Allocation strategies

- Which hole do we use for a new process?
- **First fit**: first hole that's big enough
- **Best fit**: smallest hole that's big enough
- **Worst fit**: largest hole
- Downsides?
  - → First fit: early holes get chewed up
  - → Best/worst fit: full scan
  - → **Next fit**: first fit, starting from the last allocation

Run the algorithms

- Free holes, in memory order: **100K, 500K, 200K, 300K, 600K**
- Requests, in order: **212K, 417K, 112K, 426K**
- Place each request using first fit, best fit, and worst fit
- Which strategy places all four?

External fragmentation

- Ideally, all memory gets allocated
- In reality: many holes, none big enough
  - → Although, combined they may be
- All the strategies above suffer from this
- 50-percent rule: up to 1/3 of memory may be unusable
- Compaction helps, but it’s expensive

Segmentation

- What if a process didn't need one contiguous block?
- **Segmentation**: split the address space into logical segments (code, data, stack)
- Each segment has a base and bounds, stored in a per-process segment table
- Sized to need: minimal internal fragmentation
- Still has external fragmentation

Segmentation: the fundamental limit

- Each segment is still contiguous
  - → External fragmentation just moved from processes to segments
- What if we dropped contiguity entirely?
  - → Fixed-size chunks: no external fragmentation
  - → This is the idea behind paging

What isn’t clear?

Comments? Thoughts?